

HƯỚNG DẪN BẮT ĐẦU

AURA CODE

Trợ lý lập trình tự động, không phụ thuộc mô hình AI cụ thể. Nhập yêu cầu bằng ngôn ngữ tự nhiên → đọc mã nguồn → lập kế hoạch → thực thi → kiểm chứng → báo cáo.

v0.10.0

`npm install -g aura-code`

TypeScript

MIT

github.com/milodule3-debug/aura-code

Phiên Bản Tiếng Việt

01 Trước Khi Bắt Đầu — Cài Đặt Node.js

YÊU CẦU BAN ĐẦU

Aura Code chạy trên nền Node.js. Bạn cần cài Node.js trước tiên — `npm` (trình quản lý gói của Node) đã đi kèm sẵn.

Bạn cần gì

- **Node.js phiên bản 20 trở lên** (Node 22 đã được kiểm thử và khuyến nghị)
- Một cửa sổ dòng lệnh — Terminal.app trên macOS, shell bất kỳ trên Linux, PowerShell hoặc WSL trên Windows
- Kết nối internet để cài gói và gọi các nhà cung cấp AI

macOS

- 1 Cài Homebrew nếu chưa có, sau đó cài Node:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install node
```

Linux (Ubuntu / Debian)

- 1 Dùng script cài đặt của NodeSource để có phiên bản mới — kho mặc định của Ubuntu thường quá cũ:

```
# Node 22 LTS
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Windows

- 1 Tải trình cài đặt bản LTS từ **nodejs.org** rồi chạy, hoặc dùng winget:

```
winget install OpenJS.NodeJS.LTS
```

Giao diện dòng lệnh của Aura chạy tốt nhất trong **WSL (Windows Subsystem for Linux)** hoặc Windows Terminal — `cmd.exe` thông thường hỗ trợ hạn chế cho giao diện modal tương tác.

Kiểm tra lại

```
node --version # phải hiện v20.x trở lên
npm --version
```

VÌ SAO BƯỚC NÀY QUAN TRỌNG

Mọi bước sau — cài Aura, chạy Aura, kết nối Telegram — đều phụ thuộc vào việc `node` và `npm` chạy được. Nếu một trong hai lệnh báo "not found," hãy dừng lại và xử lý ngay tại đây trước.

Aura được phát hành trên npm và khi cài sẽ tạo ra hai lệnh toàn cục: `aura` và `dic`.

1 Cài đặt toàn cục:

```
npm install -g aura-code
```

2 Kiểm tra cả hai lệnh đều hoạt động:

```
aura --version  
which aura  
which dic
```

NẾU `AURA` HOẶC `DIC` ĐỘT NHIÊN BIẾN MẤT

Liên kết npm toàn cục có thể bị mất — ví dụ khi có lệnh `npm unlink` hoặc `npm uninstall -g` chạy nhầm thư mục mà không chỉ rõ tên gói. Bản build của bạn không hề mất; chỉ là symlink toàn cục bị đứt. Liên kết lại từ thư mục dự án:

```
cd path/to/aura-code  
npm run build  
npm link
```

Nếu không có mã nguồn sẵn trên máy, chỉ cần cài lại: `npm install -g aura-code`. Sau đó chạy `aura --doctor --fix` để rà soát các vấn đề còn sót lại.

Cập nhật phiên bản mới

```
npm update -g aura-code
```

Aura cũng có thể tự kiểm tra: `aura --doctor` quét bản build, cấu hình, thư viện phụ thuộc, biến môi trường và trạng thái git của chính nó; thêm `--fix` để nó tự sửa, hoặc `--offline` để bỏ qua bước kiểm tra phiên bản qua mạng.

Aura cần ít nhất một API key từ nhà cung cấp AI trước khi có thể làm việc gì. Cách dễ nhất là dùng trình hướng dẫn thiết lập có sẵn.

- 1 Từ bất kỳ thư mục nào, chạy Aura không kèm tham số — lần đầu tiên sẽ tự mở trình hướng dẫn thiết lập tương tác:

```
aura
```

- 2 Trình hướng dẫn sẽ dẫn bạn chọn nhà cung cấp và mô hình, rồi hỏi API key (xem Phần 04 để biết cách lấy các key đầu tiên). Sau khi thiết lập xong, bạn sẽ vào giao diện dòng lệnh dạng modal của Aura.

- 3 Nếu đã đặt sẵn key qua biến môi trường, bạn có thể bỏ qua trình hướng dẫn và giao việc luôn:

```
# Giao một việc, không mở phiên tương tác
aura "add a logout button to the navbar"

# Chế độ hoàn toàn tự động — không hỏi xác nhận
aura --auto "fix the failing tests"

# Chỉ định model hoặc provider cụ thể
aura -m claude-opus-4-5-20251001 "refactor the auth module"
aura --provider openrouter -m gpt-4o "add unit tests"

# Không cần API key — chạy hoàn toàn cục bộ qua Ollama
aura --profile local -m ollama/llama3.2 "explain this codebase"
```

| Bên trong giao diện (TUI)

Giao diện dòng lệnh của Aura theo kiểu modal, giống vim: chế độ **INSERT** để gõ việc cần làm, chế độ **SCROLL** (nhấn **Esc**) để cuộn lại xem lịch sử phiên làm việc bằng **j**/**k**. Nhấn **i** hoặc **Enter** để quay lại chế độ gõ. Gõ **:help** bất cứ lúc nào để xem toàn bộ danh sách lệnh (Phần 06 có in đầy đủ).

Aura không phụ thuộc vào một mô hình AI cụ thể — hoạt động được với mọi nhà cung cấp. Dưới đây là ba điểm khởi đầu tốt: hai cổng trung gian để dùng và một nhà cung cấp trực tiếp, để bạn có nhiều lựa chọn mô hình ngay từ đầu.

1 OpenCode Go

opencode.ai/docs/go

Gói đăng ký chi phí thấp, cho phép truy cập một tập hợp mô hình mã nguồn mở đã được tuyển chọn (DeepSeek, GLM, Kimi K2, v.v.), được host để đảm bảo truy cập ổn định trên toàn cầu — đặc biệt phù hợp khi dùng từ Việt Nam, vì độ trễ và khả năng truy cập tới một số nhà cung cấp khác có thể không ổn định.

- Đăng nhập tại **opencode.ai** và đăng ký gói **OpenCode Go** (qua OpenCode Zen)
- Sao chép API key được cấp
- Trong Aura, key này được thêm dưới dạng nhà cung cấp tùy chỉnh tương thích OpenAI — qua trình hướng dẫn `:provider`, hoặc chỉnh tay trong `.aura.json` (xem khối cấu hình ở trang sau)

2 OpenRouter

openrouter.ai/keys

Một cổng trung gian duy nhất truy cập hàng trăm mô hình từ OpenAI, Anthropic, Google, Meta và nhiều hãng khác — hữu ích khi muốn thử nhiều mô hình khác nhau mà không cần xin key riêng cho từng nhà cung cấp.

- Đăng ký tại **openrouter.ai** (email, Google, hoặc GitHub)
- Nạp tiền (trả trước — vài đô la là đủ để bắt đầu)
- Vào **openrouter.ai/keys**, bấm **Create Key**, và sao chép ngay — key chỉ hiển thị một lần duy nhất
- Đặt vào biến môi trường `OPENROUTER_API_KEY` (xem bảng bên dưới)

3 Anthropic (Claude)

console.anthropic.com

Truy cập trực tiếp các mô hình Claude (Opus, Sonnet, Haiku) — nhà cung cấp mặc định/tham chiếu của chính Aura, nên có sẵn một key trực tiếp dù bạn chủ yếu dùng qua cổng trung gian.

- Đăng ký hoặc đăng nhập tại **console.anthropic.com**
- Vào mục **API Keys**, tạo key mới và sao chép lại
- Đặt vào biến môi trường `ANTHROPIC_API_KEY`

Cách A — dùng trình hướng dẫn (dễ nhất)

Bên trong Aura, chạy trình hướng dẫn thiết lập nhà cung cấp — nó sẽ hỏi nhà cung cấp, mô hình và key, rồi tự lưu lại:

```
:provider
```

Để chỉ đặt key cho nhà cung cấp đang dùng:

```
:apikey <your-key>
```

Cách B — biến môi trường

Thêm các dòng sau vào file cấu hình shell (`~/.bashrc` , `~/.zshrc`) để dùng được ở mọi phiên làm việc:

```
# ~/.bashrc hoặc ~/.zshrc
export ANTHROPIC_API_KEY=sk-ant-...
export OPENROUTER_API_KEY=sk-or-...
```

Sau đó nạp lại: `source ~/.bashrc`

BIẾN MÔI TRƯỜNG	NHÀ CUNG CẤP
ANTHROPIC_API_KEY	Các mô hình Claude
OPENAI_API_KEY	Các mô hình GPT
GOOGLE_API_KEY	Các mô hình Gemini
XAI_API_KEY	Các mô hình Grok
OPENROUTER_API_KEY	OpenRouter (tất cả mô hình)
XIAOMI_API_KEY	Xiaomi MiMo
ZHIPU_API_KEY	Zhipu GLM (Z.ai)

Cách C — cấu hình theo dự án (.aura.json)

Để cấu hình riêng cho từng dự án, hoặc để thêm nhà cung cấp tùy chỉnh tương thích OpenAI như OpenCode Go, tạo file `.aura.json` ở thư mục gốc dự án:

```
{
  "model": "claude-sonnet-4-5-20251001",
  "mode": "auto",
  "providers": [
    {
      "name": "OpenCodeGo",
      "baseUrl": "https://opencode.ai/zen/v1",
      "apiKeyEnv": "OPENCODE_GO_API_KEY",
      "prefixes": ["opencode-go/"],
      "models": [
        { "id": "opencode-go/kimi-k2.7-code", "name": "Kimi K2.7 Code", "speed":
"Fast" }
      ]
    }
  ],
  "fallbacks": ["gpt-4o-mini", "gemini-2.5-flash"]
}
```

LƯU Ý

Các nhà cung cấp tùy chỉnh (như OpenCode Go) được cấu hình dưới dạng endpoint tương thích OpenAI. Hãy kiểm tra base URL và mã model hiện tại trong tài liệu chính thức của OpenCode trước khi dùng ví dụ trên — endpoint của nhà cung cấp có thể thay đổi theo thời gian.

Bot Telegram của Aura cho phép bạn giao việc, nhận trả lời bằng giọng nói, và điều khiển máy tính từ xa qua điện thoại.

- 1 Trong Telegram, nhắn tin cho **@BotFather** và gửi lệnh `/newbot`. Làm theo hướng dẫn để đặt tên bot; BotFather sẽ cấp cho bạn một bot token — một chuỗi dài dạng `123456:ABC-DEF...`.

- 2 Trong Aura, chỉ cần yêu cầu bằng lời tự nhiên, ví dụ:

```
connect telegram
```

Aura sẽ hỏi lại bạn bot token. Dán vào khi được hỏi.

- 3 Sau khi kết nối, nhắn tin cho bot của bạn từ điện thoại. Các lệnh bot hữu ích khi đã chạy: `/status`, `/stop`, `/approve-all`, `/safety_off`, `/ls`, `/read`, `/run`, `/search`.

BẢO MẬT BOT TOKEN NHƯ MẬT KHẨU

Bất kỳ ai có bot token đều có thể gửi lệnh điều khiển máy tính của bạn qua bot. Tuyệt đối không đăng token lên nhóm chat công khai, repo, hay issue tracker. Nếu bị lộ, thu hồi ngay lập tức trong @BotFather (`/revoke`) và tạo token mới.

06 Kết Nối Gmail, Google Drive & Các Dịch Vụ Khác

TÍCH
HỢP

Cùng một cách làm hội thoại áp dụng cho các dịch vụ khác — bạn yêu cầu, Aura hỏi lại thông tin xác thực cần thiết.

1 Chỉ cần nói với Aura muốn kết nối gì, bằng ngôn ngữ tự nhiên:

```
connect gmail
connect google drive
```

Aura sẽ hỏi thông tin xác thực cần thiết hoặc dẫn bạn qua bước xác thực mà dịch vụ đó yêu cầu — có thể là dán một token, hoặc mở trình duyệt để đăng nhập OAuth, tùy theo từng dịch vụ.

NÓI RÕ DỊCH VỤ NÀO

"Kết nối Gmail" và "kết nối Google Drive" là hai kết nối riêng biệt với hai thông tin xác thực khác nhau, dù cả hai đều là sản phẩm của Google — đừng nghĩ kết nối một cái là tự động kết nối luôn cái kia.

NẾU KẾT NỐI KHÔNG HOẠT ĐỘNG NHƯ MONG ĐỢI

Các tích hợp là phần thay đổi nhanh nhất trong Aura. Nếu một dịch vụ không kết nối như bạn nghĩ, chạy `:help` để xem những gì hiện đang được hỗ trợ, hoặc `aura --doctor` để kiểm tra môi trường và cấu hình xem có thiếu gì không. Đừng đoán mò một cờ lệnh hay lệnh không có trong `:help` — thay vào đó hỏi thẳng Aura ("làm sao để kết nối X") và để nó cho biết hiện tại nó hỗ trợ gì thật sự.

07 Sức Mạnh Thực Sự — Các Lệnh Phức Tạp

QUY TRÌNH LÀM VIỆC

Khi đã quen với các thao tác cơ bản, đây là những lệnh khiến Aura thực sự khác biệt so với một trợ lý chat thông thường.

| :machina — thực thi tự kiểm chứng

```
:machina refactor the payment module to use the new API client
```

Chạy việc, sau đó tự kiểm chứng kết quả và tự động thử lại nếu kiểm chứng thất bại — phù hợp nhất cho những việc bạn không muốn phải ngồi canh.

| :council — đánh giá song song bởi nhiều chuyên gia

```
:council should we migrate this service to gRPC?
```

Khởi chạy 2-3 agent chuyên gia song song (chỉ đọc, không sửa) để xem xét câu hỏi từ nhiều góc độ khác nhau, sau đó tổng hợp thành một câu trả lời.

| :ecclesia — nghiên cứu độc lập chuyên sâu

```
:ecclesia evaluate our current rate-limiting strategy
:ecclesia compare Postgres vs. SQLite for our use case --panel gpt-4o --seats 5
```

Năm agent nghiên cứu độc lập tự tìm hiểu riêng, sau đó tổng hợp thành một kết luận và lưu vào `council/*.md` và `council/*.html`.

| :workflow — quy trình nhiều bước có tên

```
:workflow release "run tests" "bump version" "update changelog" "push and tag"
```

Chạy từng bước theo thứ tự, mỗi bước là một task riêng. Nếu một bước thất bại, có thể tiếp tục sau mà không cần lặp lại các bước đã thành công:

```
:resume-workflow <id>
```

| --orchestrate — chế độ nhiều agent

```
aura --orchestrate "add multi-tenant support to the API layer"
```

Chia nhỏ công việc theo quy trình Researcher → Coder → Reviewer, thay vì một agent duy nhất làm tuần tự toàn bộ.

| :dream / :rem / :mine — công cụ bộ nhớ

```
:dream          # gộp các episode gần đây thành một bản ghi nhớ
:rem            # xem bộ nhớ đã hợp nhất / bản ghi nhớ gần nhất
:mine --refine   # gom cụm episode thành training-data/*.jsonl
```

Các lệnh này giúp Aura xây dựng bộ nhớ lâu dài qua các phiên làm việc, thay vì bắt đầu lại từ đầu mỗi lần.

Chạy các lệnh này từ shell thông thường, trước hoặc không cần mở giao diện TUI.

– Cài đặt & phiên bản –

LỆNH	CHỨC NĂNG
<code>npm install -g aura-code</code>	Cài Aura toàn cục
<code>npm update -g aura-code</code>	Cập nhật lên phiên bản mới nhất
<code>aura --version</code>	In phiên bản đã cài

– Giao việc –

LỆNH	CHỨC NĂNG
<code>aura</code>	Mở giao diện TUI / trình hướng dẫn lần đầu
<code>aura "task"</code>	Giao một việc, không mở phiên tương tác
<code>aura --auto "task"</code>	Hoàn toàn tự động — không hỏi xác nhận
<code>aura --readonly "task"</code>	Chỉ phân tích, không chỉnh sửa
<code>aura --orchestrate "task"</code>	Nhiều agent: Researcher → Coder → Reviewer
<code>aura --verify "task"</code>	Kiểm tra sau khi hoàn thành, tự thử lại
<code>aura -m <id> "task"</code>	Chỉ định model cụ thể
<code>aura --provider <name> "task"</code>	Chỉ định provider cụ thể
<code>aura --profile local -m ollama/<model></code>	Chạy hoàn toàn cục bộ qua Ollama, không cần API key

– Quy trình & chẩn đoán –

LỆNH	CHỨC NĂNG
<code>aura --workflow <name> "b1" "b2"</code>	Tạo và chạy quy trình nhiều bước có tên
<code>aura --resume-workflow <id></code>	Tiếp tục quy trình bị tạm dừng/lỗi
<code>aura --workflows</code>	Liệt kê toàn bộ quy trình đã lưu
<code>aura --doctor</code>	Quét build, cấu hình, thư viện, môi trường, git của Aura
<code>aura --doctor --fix</code>	Quét và tự động sửa lỗi
<code>aura --doctor --offline</code>	Tương tự, bỏ qua kiểm tra phiên bản qua mạng

– Cờ chịu lỗi (resilience) –

CỜ LỆNH	CHỨC NĂNG
<code>--rate-limit-rpm <n></code>	Giới hạn số request mỗi phút (0 = không giới hạn)
<code>--rate-limit-tpm <n></code>	Giới hạn số token mỗi phút (chỉ Google)
<code>--max-retries <n></code>	Số lần thử lại tối đa khi gặp lỗi 429/5xx (mặc định 5)
<code>--fallback <model></code>	Model dự phòng khi model chính hết lượt thử lại (có thể lặp lại)

09

Danh Mục Lệnh — Bên Trong Aura

LỆNH COLON TRONG TUI

Gõ các lệnh này khi đã ở trong giao diện TUI của Aura. Đây là danh sách chính thức từ `:help` — gõ `:help` bất cứ lúc nào để xem trực tiếp.

– Phiên làm việc –

<code>:id</code>	Hiện ID phiên hiện tại
<code>:sessions</code>	Liệt kê tất cả phiên đã lưu
<code>:resume [id]</code>	Tiếp tục phiên gần nhất hoặc chỉ định
<code>:new</code>	Bắt đầu phiên mới
<code>:history</code>	Hiện số lượt trao đổi
<code>:clear-history</code>	Xóa lịch sử, giữ nguyên ID phiên
<code>:save [title]</code>	Đổi tên / lưu phiên
<code>:delete <id></code>	Xóa một phiên đã lưu

– Model / API –

<code>:model</code>	Chọn model tương tác
<code>:model <id></code>	Chuyển sang model cụ thể
<code>:models</code>	Liệt kê các model khả dụng
<code>:provider</code>	Trình hướng dẫn thiết lập provider
<code>:apikey <key></code>	Đặt key cho provider hiện tại

– Quy trình làm việc –

<code>:workflows</code>	Liệt kê quy trình đã lưu
<code>:workflow <name> "b1"...</code>	Tạo & chạy một quy trình
<code>:resume-workflow <id></code>	Tiếp tục quy trình bị dừng/lỗi
<code>:q add <prompt></code>	Thêm việc vào hàng đợi
<code>:q list</code>	Liệt kê việc trong hàng đợi
<code>:q run <n></code>	Chạy việc số #n trong hàng đợi
<code>:q drop <n></code>	Xóa việc số #n khỏi hàng đợi
<code>:q clear</code>	Xóa toàn bộ hàng đợi
<code>:machina <task></code>	Thực thi tự kiểm chứng
<code>:council <task></code>	2-3 chuyên gia song song
<code>:ecclesia <topic></code>	5 nhà nghiên cứu độc lập

– Bộ nhớ / Phụ trợ –

<code>:dream [full]</code>	Gộp episode thành bộ nhớ
<code>:rem</code>	Xem bộ nhớ đã hợp nhất / gần nhất
<code>:mine [--refine]</code>	Khai thác episode tìm quy luật
<code>:research <topic></code>	Nghiên cứu nhiều bước
<code>:confess</code>	Tự phát hiện & báo cáo bất thường
<code>:confessions</code>	Liệt kê các báo cáo bất thường
<code>:btw <question></code>	Câu hỏi phụ nhanh, không ảnh hưởng lịch sử

– Giọng nói –

<code>:speak</code>	Bật/tắt đọc to câu trả lời
---------------------	----------------------------

– An toàn –

<code>:approve</code>	Bật/tắt tự động phê duyệt
<code>:approve all</code>	Phê duyệt mọi thứ trong phiên này
<code>:approve off</code>	Bật lại yêu cầu xác nhận

– Ngữ cảnh / Thống kê –

<code>:context</code>	Hiện ngữ cảnh dự án đã nạp
<code>:graph [refresh]</code>	Sơ đồ tri thức mã nguồn
<code>:plans</code>	Liệt kê kế hoạch thực thi đã lưu
<code>:viz, :dashboard</code>	Mở bảng điều khiển bộ nhớ
<code>:doctor [--fix]</code>	Quét lỗi của chính Aura
<code>/stats, /usage</code>	Số token + chi phí đã dùng trong phiên
<code>/context</code>	Bảng theo dõi tình trạng ngữ cảnh
<code>/clear, /reset</code>	Đặt lại thống kê sử dụng

– Chung –

<code>:quit, :q, /exit</code>	Thoát Aura
<code>Esc</code>	Vào chế độ SCROLL
<code>i / Enter</code>	Quay lại chế độ INSERT
<code>j / k</code>	Cuộn xuống / lên (chế độ SCROLL)

Bây giờ bạn đã có Node.js, Aura Code, ba API key, và Telegram được kết nối — đầy đủ mọi thứ cần thiết để giao những việc thật, tự động, trên chính mã nguồn của bạn.

Tóm tắt nhanh

- Cài Node.js 20 trở lên, sau đó `npm install -g aura-code`
- Chạy `aura` và hoàn tất trình hướng dẫn thiết lập, hoặc đặt key qua biến môi trường
- Lấy key từ OpenCode Go, OpenRouter, và Anthropic (Phần 04)
- Yêu cầu Aura bằng ngôn ngữ tự nhiên để kết nối Telegram, Gmail, Drive, hoặc bất kỳ dịch vụ nào khác được hỗ trợ
- Khi đã sẵn sàng nâng cao hơn, dùng `:machina`, `:council`, `:ecclesia`, và `:workflow`

NẾU CÓ SỰ CỐ

Trước tiên chạy `aura --doctor --fix`. Nếu `aura` hoặc `dic` hoàn toàn không nhận lệnh nữa, gần như luôn là do mất liên kết npm toàn cục, không phải do build bị hỏng — liên kết lại từ mã nguồn bằng `npm link`, hoặc cài lại bằng `npm install -g aura-code`.

github.com/milodude3-debug/aura-code · Giấy phép MIT · Được xây dựng bởi các AI agent, điều phối bởi con người.